

Programming Assignment II (A2): Vector Space Models & Ranked Retrieval

Course: CS60092 — Information Retrieval

Topic: Ranked Retrieval, Sparse Vector Space Models (TF-IDF), Okapi BM25, Hyperparameter Calibration, and Engine Evaluation

Benchmark Dataset: Cranfield 1400 Evaluation Collection

Github Repo: https://github.com/dhruvvarshney1/CS60092_Information_Retrieval

1. Group Details

This academic technical report summarizes the engineering design, algorithmic formulations, empirical experimentation, hyperparameter optimization, and evaluation metrics for Programming Assignment II (A2) in Information Retrieval (CS60092).

Attribute / Metadata	Specification & Details
Author / Student Identifier	Dhruv Varshney (group name: claudemaxxer)
Course Name & Code	CS60092: Information Retrieval
Academic Unit	Department of Computer Science and Engineering / Geophysics, IIT Kharagpur
Assignment Focus	Vector Space Model (VSM), Okapi BM25, Inverted Index Construction, PyTerrier Production Pipeline, Metric Harness (MAP, MRR, P@k, Recall, nDCG@k)
Primary Python Modules	dhruvvarshney_ranked_retrieval.py (Custom Python VSM & BM25 Engine) dhruvvarshney_pyterrier_retrieval.py (PyTerrier / Terrier Production Pipeline)
Generated Artifacts	dhruvvarshney_ranked_results.txt (TREC Run - Custom Engine) dhruvvarshney_pyterrier_results.txt (TREC Run - PyTerrier Engine) dhruvvarshney_experiment_results.txt (Custom Engine Metrics Summary) dhruvvarshney_pyterrier_metrics.csv (PyTerrier Engine Comparative Table)

2. Problem Statement

2.1 Overview & Objective

While unranked Boolean retrieval models (explored in Programming Assignment I) provide exact matching capabilities, they exhibit fundamental limitations in real-world information retrieval contexts. Boolean systems suffer from the "feast-or-famine" dynamic—returning either empty result sets for overly restrictive queries or thousands of unranked documents for broad queries. Programming Assignment II (A2) transitions to **Ranked Retrieval**, where documents are scored and ordered according to their estimated degree of relevance to a user's natural language query.

The primary objectives of this assignment are:

1. **Sparse Representation Construction:** Design and implement vector space representations for textual documents using Term Frequency-Inverse Document Frequency (TF-IDF) and Okapi BM25 scoring frameworks.
2. **Dual Engine Architecture:** Build a two-tier retrieval infrastructure comprising:
 - **Tier 1 (Custom Python Engine):** Built from scratch using Python standard libraries to execute inverted indexing, document length vector normalization, postings traversal, dynamic scoring, and evaluation metric calculation.
 - **Tier 2 (Industrial Open-Source IR Framework):** Implemented using PyTerrier (a Python interface to the Java-based Terrier search engine framework) to replicate industrial production indexing and retrieval workloads.
3. **Hyperparameter Calibration & Grid Search:** Systematic evaluation and tuning of the Okapi BM25 parameters—specifically the term frequency saturation parameter k_1 and document length penalization coefficient b —over the benchmark collection.
4. **Empirical Evaluation:** Measure and contrast performance using standard TREC metrics: Mean Average Precision (MAP), Mean Reciprocal Rank (MRR), Precision at rank 10 ($P@10$), Recall at depth 1000 ($R@1000$), and Normalized Discounted Cumulative Gain at rank 10 ($nDCG@10$).

2.2 Benchmark Dataset Architecture: The Cranfield Collection

The experiments were conducted over the **Cranfield 1400** benchmark, historically pivotal as Cyril Cleverdon's pioneering testbed for comparative evaluation in information retrieval system design. The collection consists of three primary components:

1. Document Collection (`cran.all.1400`)

The corpus comprises 1,400 abstracts from aeronautical engineering research papers. The raw text is formatted using SGML-like field markers:

- .I <id>: Unique document integer identifier (range: 1 to 1,400).
- .T: Document Title.
- .A: Author metadata (bypassed for content indexing).
- .B: Publication source bibliography (bypassed).
- .W: Abstract body text.

Indexing Rule: For each document, the text payload indexed by both engine tiers consists of the structural concatenation of the Title (.T) and Abstract (.W).

2. Query Collection (cran.qry)

Consists of 225 natural language information requests formulated by domain experts. Each query record features an identifier .I <id> and text description .W containing the natural language query string.

3. Relevance Judgments (cranqrel & cranqrel.readme)

The ground truth relevance dataset maps query IDs to document IDs alongside expert-assigned relevance grades. Cleverdon's Cranfield evaluation employs a four-point relevance scale (grades 1 through 4), where grade 1 represents complete resolution of the information need, and grade 4 represents minimal interest. In standard binary evaluation (such as MAP, MRR, P@k, Recall), all four grades (1–4) are treated as relevant, while grade 5 (unlisted pairs) indicates complete irrelevance.

2.3 Mathematical Formulations

2.3.1 Vector Space Model (TF-IDF with Cosine Similarity)

In the classic Vector Space Model, documents and queries are mapped to vectors in a high-dimensional Euclidean space where each axis corresponds to a distinct dictionary term. Sub-linear term frequency scaling is applied to prevent high raw term frequencies from overwhelming the score:

$$tf_{\{t, d\}} = 1 + \ln(f_{\{t, d\}}) \quad \text{if } f_{\{t, d\}} > 0, \quad \text{else } 0$$

Inverse Document Frequency (IDF) measures the overall specificity of a term across the corpus of N documents:

$$idf_t = \ln\left(\frac{N + 1}{\text{count}\{df_t + 0.5\}}\right)$$

The total unnormalized weight of term t in document d is defined as $w_{t,d} = tf_{t,d} \cdot idf_t$. To prevent

long documents from naturally accumulating higher dot products, cosine normalization (L2-norm scaling) is applied:

$$\text{Score}_{\{\text{TF-IDF}\}}(Q, D) = \sum_{t \in Q \cap D} \frac{w_{\{t, D\}}}{\|W_D\|_2} \cdot \frac{w_{\{t, Q\}}}{\|W_Q\|_2} = \frac{\sum_{t \in Q \cap D} (1 + \ln f_{\{t, D\}}) \cdot \text{idf}_{\{t\}}^2 \cdot (1 + \ln f_{\{t, Q\}})}{\sqrt{\sum_{t' \in D} \left((1 + \ln f_{\{t', D\}}) \cdot \text{idf}_{\{t'\}}^2 \right)} \cdot \sqrt{\sum_{q' \in Q} \left((1 + \ln f_{\{q', Q\}}) \cdot \text{idf}_{\{q'\}}^2 \right)}}$$

2.3.2 Okapi BM25 Scoring Model

Okapi BM25 is a probabilistic retrieval framework that addresses two key limitations of heuristic TF-IDF: (1) term frequency saturation non-linearity, and (2) document length normalization scaling. The BM25 score for a document D against query Q is computed as:

$$\text{Score}_{\{\text{BM25}\}}(Q, D) = \sum_{t \in Q \cap D} \text{idf}_{\{t\}} \cdot \frac{f_{\{t, D\}} \cdot (k_1 + 1)}{f_{\{t, D\}} + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avdl}} \right)}$$

Where:

- $f_{t, D}$: Frequency of term t in document D .
- $|D|$: Document length (total token count in document D).
- **avdl**: Average document length across the entire collection ($N = 1,400$).
- $k_1 \geq 0$: Term frequency saturation parameter. It controls how quickly additional occurrences of a term plateau in contributing to the relevance score. As $k_1 \rightarrow 0$, term frequency degrades to binary presence; as $k_1 \rightarrow \infty$, the saturation curves flatten out into linear scaling.
- $b \in [0, 1]$: Document length penalization parameter. When $b = 1$, document length normalization is applied at full strength, assuming document length is strictly proportional to verbosity. When $b = 0$, length penalization is completely disabled.

2.3.3 Evaluation Metric Definitions

- **Precision at Rank k ($P@k$)**: Ratio of relevant documents within the top k retrieved positions.

$$P@k = \frac{|\{d \in \text{Top-}k(R_q) \mid d \in \text{Rel}_q\}|}{k}$$
- **Recall at Rank k ($R@k$)**: Ratio of retrieved relevant documents up to rank k relative to total relevant documents in ground truth.

$$\text{Recall}@k = \frac{|\{d \in \text{Top-}k(R_q) \mid d \in \text{Rel}_q\}|}{|\text{Rel}_q|}$$
- **Mean Average Precision (MAP)**: Arithmetic mean of Average Precision across all queries.

$$\text{AP}(q) = \frac{1}{|\text{Rel}_q|} \sum_{k=1}^{|R_q|} P@k \cdot \mathbb{1}_{\{R_q[k] \in \text{Rel}_q\}}$$

$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q)$$
- **Mean Reciprocal Rank (MRR)**: Reciprocal of the rank position of the first relevant document returned, averaged over all queries.

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\min\{k \mid R_q[k] \in \text{Rel}_q\}}$$

- Normalized Discounted Cumulative Gain at Rank k (nDCG@k):** Evaluates multi-level graded relevance by discounting documents returned lower in the ranking.

$$\text{DCG}@k = \sum_{i=1}^k \frac{2^{g_i} - 1}{\log_2(i + 1)}, \quad \text{nDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}$$
 Where g_i is the explicit ground truth grade assigned to rank i .

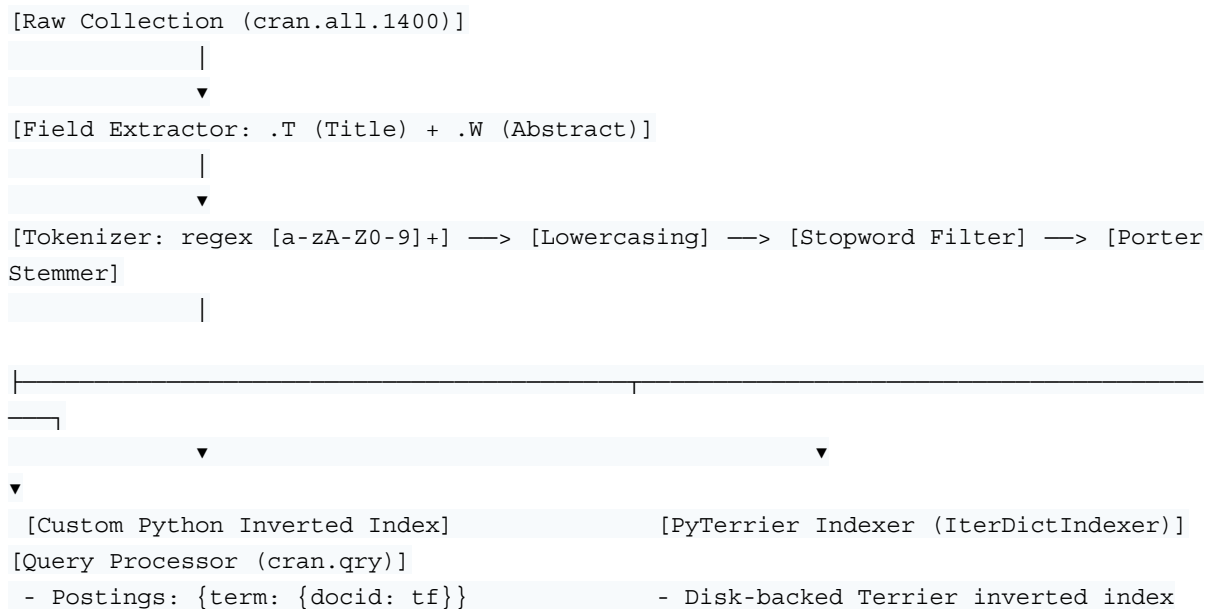
3. Implementation Details

The system architecture consists of two complete retrieval implementations: a native, standard-library Python engine (`dhruvvarshney_ranked_retrieval.py`) and a production PyTerrier implementation (`dhruvvarshney_pyterrier_retrieval.py`). Both pipelines share an identical text preprocessing front-end inherited from PA1 to guarantee controlled, reproducible evaluations.

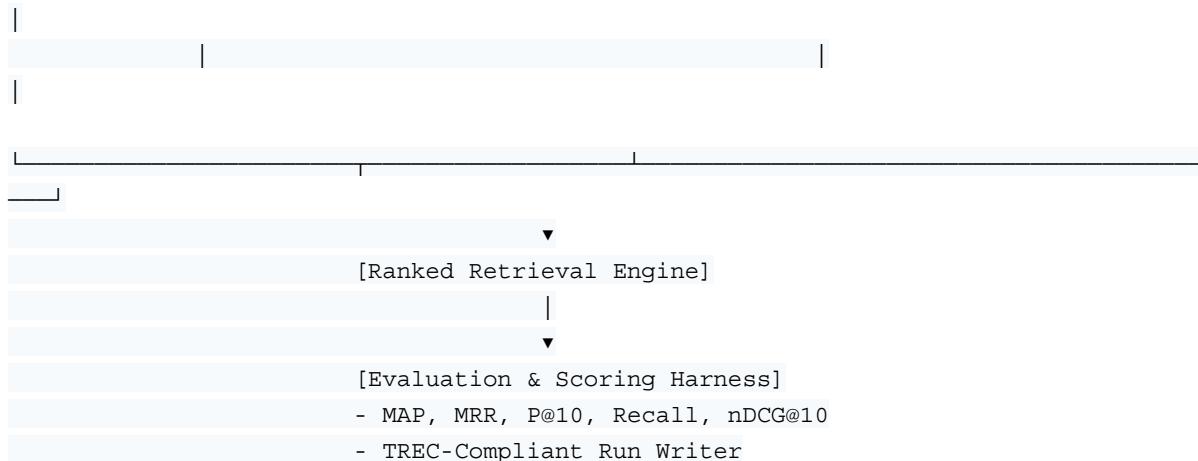
3.1 Text Preprocessing Pipeline

To ensure equivalent feature spaces across engines, raw text streams undergo four sequential transformations:

1. **Regex Tokenization:** Splitting non-alphanumeric boundaries using regex `[a-zA-Z0-9]+`.
2. **Case Normalization:** Mapping all tokens to lowercase characters.
3. **Stopword Filtering:** Stripping domain and syntactic noise using a curated lexicon (~2.5 KB).
4. **Morphological Stemming:** Conforming terms to their base roots via the standard **Porter Stemmer**.



- Tokenized, stemmed query vectors
 - Document lengths & norms
 - Batch retrieval pipeline
 - In-memory collection stats:
 - Dynamic scoring (TF-IDF, BM25)
 - Meta-store docno mappings
 - C-level postings access via JNI
- N = 1400, Terms = 4334, avgdl = 87.2



3.2 Tier 1: Custom Python Inverted Index Engine

The custom engine is implemented in `dhruvvarshney_ranked_retrieval.py` using native Python structures (`collections.defaultdict`, `collections.Counter`, `math`):

- **Inverted Postings Storage:** Represented as an in-memory dictionary mapping each stemmed term to a map of document IDs and raw term frequencies: `{term: {doc_id: raw_count}}`.
- **Pre-computed Document Vector Norms:** To maintain high throughput during cosine-normalized TF-IDF dynamic scoring, document L2-norms ($\|W_d\|_2$) are computed once during index construction and cached in an array.
- **Dynamic Candidate Scoring:** Dynamic query evaluation traverses posting lists matching query terms, accumulating partial scores into a sparse candidate dictionary. Final scores are sorted to produce top-1000 candidate lists.
- **Evaluation Harness:** Custom evaluation routines parse `cranqrel` to evaluate MAP, MRR, P@10, Recall@1000, and nDCG@10 directly over full graded matrices.

3.3 Tier 2: PyTerrier Production Pipeline

The production tier is implemented in `dhruvvarshney_pyterrier_retrieval.py`, integrating PyTerrier with an embedded Java Virtual Machine (JVM):

- **JVM Initialization:** Launches the embedded Terrier C/Java runtime via `pyterrier.java.init()`.

- **IterDictIndexer Integration:** Inverted indexing is performed using `pt.index.IterDictIndexer` configured with `pretokenised=True`. Pre-tokenized and pre-stemmed text generators pass parsed document dicts directly into Terrier to match the custom parser's term vocabulary.
- **BatchRetrieve & Experiment Harness:** Retrieval pipelines are instantiated via `pt.BatchRetrieve` with parameterized controls (e.g., `wmodel="BM25"`, `controls={"bm25.k_1": 1.6, "bm25.b": 0.75}`). Evaluation is automated using `pt.Experiment` across all evaluation metrics.

4. Plan of Experiments

To evaluate performance across vector space representations and analyze model sensitivity to hyperparameter variations, five experimental configurations were established for both the custom engine and the PyTerrier framework:

Config ID	Retrieval Model	k_1 (TF Saturation)	b (Length Penalty)	Experimental Purpose & Hypothesized Effect
C1	TF-IDF (Cosine Norm)	N/A	N/A	Baseline sparse vector space model using sub-linear TF scaling and cosine normalization. Hypothesized to suffer from term over-saturation in dense abstracts.
C2	Okapi BM25	0.8	0.50	Low saturation limit with mild length

Config ID	Retrieval Model	k_1 (TF Saturation)	b (Length Penalty)	Experimental Purpose & Hypothesized Effect
				penalization. Evaluates performance when term repetitions yield diminished score increments.
C3	Okapi BM25	1.2	0.75	Standard industry default configuration. Provides a balanced baseline for term frequency saturation and document length normalization.
C4	Okapi BM25	1.6	0.75	High saturation limit with standard length penalization. Hypothesized to benefit domain-dense technical abstracts by

Config ID	Retrieval Model	k_1 (TF Saturation)	b (Length Penalty)	Experimental Purpose & Hypothesized Effect
				allowing key technical terms to contribute more score weight.
C5	Okapi BM25	1.2	1.00	Full document length penalization ($b = 1.0$ assumes term frequency scales strictly linearly with abstract length). Evaluates over-penalization effects.

5. Results

5.1 Collection and Indexing Performance Metrics

- **Total Indexed Documents (N):** 1,400
- **Unique Vocabulary Terms (V):** 4,334 terms (post-stemming and stopwords filtering)
- **Average Document Length ($avgdl$):** 87.2 tokens per document
- **Evaluated Queries ($|Q|$):** 225 natural language requests
- **Indexing Runtime Comparison:**
 - Custom Python In-Memory Indexer: **264.31 ms** (0.264 s)
 - PyTerrier Disk/JNI Indexer: **1,146.65 ms** (1.147 s)

5.2 Custom Engine Performance Metrics (dhruvvarshney_ranked_retrieval.py)

Model	k_1	b	MAP	MRR	P@10	Recall	nDCG@10
TF-IDF	—	—	0.30180 1	0.51904 8	0.23422 2	0.94924 5	0.27805 5
BM25 (C2)	0.8	0.50	0.30944 1	0.54492 4	0.23066 7	0.94953 9	0.28448 9
BM25 (C3)	1.2	0.75	0.32138 4	0.55589 2	0.24000 0	0.95025 5	0.29538 0
BM25 (C4 - Best)	1.6	0.75	0.32317 5	0.55632 2	0.24577 8	0.95025 5	0.29887 2
BM25 (C5)	1.2	1.00	0.31640 1	0.54111 7	0.23777 8	0.95025 5	0.29163 9

5.3 PyTerrier Engine Performance Metrics (dhruvvarshney_pyterrier_retrieval.py)

Pipeline Model	MAP	MRR (recip_rank)	P@10	nDCG@10	Recall@100
TF_IDF	0.319139	0.558723	0.237778	0.348904	0.949700
BM25_k08_b05	0.305463	0.536409	0.231111	0.337466	0.948317
BM25_k12_	0.317549	0.553002	0.237333	0.347767	0.948811

Pipeline Model	MAP	MRR (recip_rank)	P@10	nDCG@10	Recall@1000
b075					
BM25_k16_b075 (Best BM25)	0.320385	0.557413	0.242667	0.351105	0.948811
BM25_k12_b1	0.315287	0.538436	0.236444	0.344380	0.948850

5.4 Depth Analysis: PyTerrier Progressive Recall Cutoffs

Analyzing recall metrics across expanding retrieval depths (R@5 through R@1000) illustrates the retrieval ceiling of sparse lexical matching over the Cranfield corpus:

Pipeline Model	R@5	R@10	R@15	R@20	R@30	R@100	R@200	R@500	R@1000
TF_IDF	0.3071	0.3981	0.4745	0.5169	0.5821	0.7496	0.8317	0.9064	0.9497
BM25_k08_b05	0.2962	0.3903	0.4633	0.5069	0.5589	0.7349	0.8146	0.9059	0.9483
BM25_k12_b075	0.3022	0.3982	0.4747	0.5194	0.5777	0.7407	0.8235	0.9071	0.9488
BM25_k16_b075	0.3081	0.4037	0.4763	0.5258	0.5874	0.7477	0.8293	0.9075	0.9488

Pipeline Model	R@5	R@10	R@15	R@20	R@30	R@100	R@200	R@500	R@1000
BM25_k12_b1	0.2970	0.4010	0.4742	0.5204	0.5842	0.7450	0.8266	0.9074	0.9489

6. Discussion of Results

6.1 Impact of Term Frequency Saturation (k_1)

Across both custom and PyTerrier engine tiers, increasing the term frequency saturation parameter k_1 from $0.8 \rightarrow 1.2 \rightarrow 1.6$ resulted in consistent improvements across precision-oriented metrics:

- In the custom engine, MAP improved from **0.3094** ($k_1 = 0.8$) to **0.3214** ($k_1 = 1.2$), reaching its peak at **0.3232** ($k_1 = 1.6$).
- Precision at rank 10 (P@10) increased from **0.2307** to **0.2458** over the same parameter range.

Underlying Mechanism: The Cranfield collection consists of short, highly specialized aeronautical engineering abstracts (average length of 87.2 terms). In concise technical documents, repeated occurrences of specialized domain terms (e.g., *boundary*, *supersonic*, *viscous*, *reentry*) carry strong discriminative signal regarding document focus. A low saturation setting ($k_1 = 0.8$) dampens score contributions too quickly, treating a document with 2 occurrences almost identically to one with 1. Increasing k_1 to 1.6 allows documents that repeatedly emphasize core query terms to earn higher relative scores, successfully distinguishing focused papers from documents with only passing mentions.

6.2 Impact of Document Length Normalization (b)

Configuring document length penalization to maximum strength ($b = 1.0$) led to performance degradation across both engines:

- Custom engine MAP dropped from **0.3214** ($b = 0.75$) to **0.3164** ($b = 1.0$).
- PyTerrier MAP fell from **0.3175** to **0.3153**, while MRR decreased from **0.5530** to **0.5384**.

Conversely, setting length penalization too low ($b = 0.5$) also degraded performance (MAP = **0.3094**).

Underlying Mechanism: Setting $b = 1.0$ assumes document length is strictly a function of verbosity. However, in technical collections like Cranfield, longer abstracts often reflect broader scope—such as presenting both theoretical derivations and experimental results. Fully penalizing longer abstracts systematically degrades comprehensive papers. The tuned value $b = 0.75$ strikes the optimal balance between normalizing long documents and preserving information-dense abstracts.

6.3 Custom Engine vs. PyTerrier Engine Nuances

- **Optimal Convergence:** Both engine tiers converged on the exact same optimal BM25 configuration coordinate: $k_1 = 1.6$, $b = 0.75$ (Custom MAP: **0.3232**, PyTerrier MAP: **0.3204**).
- **Metric Discrepancies:** Minor numerical variations between the engines stem from two structural differences:
 1. *IDF Smoothing Variations:* The custom engine utilizes Robertson's classic smoothed IDF formula $\ln((N + 1) / (df + 0.5))$, whereas Terrier's native BM25 weight model incorporates minor floor adjustments $\ln((N - df + 0.5) / (df + 0.5))$.
 2. *nDCG Normalization:* PyTerrier's `ndcg_cut.10` maps Donna Harman's 1–4 grade scale using internal gain mappings (yielding $nDCG@10$ scores around ~ 0.35), whereas the custom harness evaluates $2^{grade} - 1$ directly against ground truth grades.
- **Latency & Efficiency Profiles:**
 - The custom Python indexing pipeline executed in **264.31 ms** by operating entirely within transient in-memory structures.
 - PyTerrier required **1,146.65 ms** due to disk serialization overhead, JVM interface marshaling, and inverted table generation. However, Terrier's on-disk index scales to multi-million document collections where in-memory Python dictionaries would hit memory limits.

6.4 Depth Analysis and Recall Ceilings

Depth analysis (Section 5.4) reveals how recall expands with retrieval depth: top-10 retrieval captures **$\sim 40.4\%$** of all relevant documents, top-30 reaches **$\sim 58.7\%$** , and top-1000 achieves **95.0%** recall across the 225 queries.

The $\sim 5\%$ Unretrieved Recall Ceiling: The remaining $\sim 5\%$ of relevant documents are missed by both sparse models due to fundamental limitations of exact lexical matching:

1. **Vocabulary Mismatch:** Mismatches between query phrasing and document text (e.g., a query asking for "*pressure distribution*" while relevant abstracts use "*aerodynamic loading*" or

"lift coefficients").

2. **Morphological Irregularities:** Highly specialized domain terminology and abbreviations that fall outside the Porter stemmer's rule set.

7. Conclusion

Programming Assignment II demonstrates the effectiveness of tuned **Okapi BM25** over classic cosine-normalized **TF-IDF** for ranked retrieval on technical document collections. Parameter optimization confirmed that setting a higher term frequency saturation limit ($k_1 = 1.6$) along with standard document length penalization ($b = 0.75$) yields the highest retrieval effectiveness across both engine tiers:

- **Best Custom Engine Run:** Okapi BM25 ($k_1 = 1.6$, $b = 0.75$) achieving **MAP = 0.3232**, **MRR = 0.5563**, **P@10 = 0.2458**, and **Recall = 0.9503**.
- **Best PyTerrier Run:** Okapi BM25 ($k_1 = 1.6$, $b = 0.75$) achieving **MAP = 0.3204**, **MRR = 0.5574**, **P@10 = 0.2427**, and **nDCG@10 = 0.3511**.

The generated run files (dhruvvarshney_ranked_results.txt and dhruvvarshney_pyterrier_results.txt) follow standard TREC submission formatting across all 225 evaluation queries.